# Node.js

Complete Beginner's Guide

From zero to your first web server

| | |
|---|---|
| ■ **Level** | Absolute Beginner |
| ■ **Read** | ~30 minutes |
| ■ **Covers** | Install · Core concepts · First server · npm |
| ■ **Goal** | Build & run your first Node.js application |

## What is Node.js?

Node.js is an open-source, cross-platform **JavaScript runtime environment** that executes JavaScript code *outside* of a web browser. Created by Ryan Dahl in 2009, it is built on Chrome's V8 JavaScript engine and uses an **event-driven, non-blocking I/O model**, making it lightweight and efficient — perfect for data-intensive real-time applications.

### Why learn Node.js?

- **Same language everywhere** — write JavaScript on both front-end and back-end.
- **Huge ecosystem** — npm hosts over 2 million packages.
- **Fast & scalable** — non-blocking I/O handles thousands of concurrent connections.
- **In-demand skill** — used by Netflix, LinkedIn, Uber, PayPal and many more.
- **Active community** — extensive documentation, tutorials and Stack Overflow answers.

■ **Tip:** Node.js is NOT a programming language. It is a runtime that lets you run JavaScript on a server.

# ■ Installation & Setup

## Installing Node.js

The easiest way to install Node.js is from the official website. Always choose the **LTS (Long-Term Support)** version for stability.

| OS | Method | Command / Steps |
|---|---|---|
| **Windows** | Installer | Download .msi from nodejs.org → run installer |
| **macOS** | Homebrew | brew install node |
| **Ubuntu** | apt | sudo apt install nodejs npm |
| **Any OS** | nvm | curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh \| bash |

### Verify your installation:

```
node --version    # e.g. v20.11.0
npm  --version    # e.g. 10.2.4
```

## Your First Node.js Script

Create a file called **hello.js** and add the following code. Then run it from your terminal.

```
// hello.js
console.log('Hello, Node.js!');

const name = 'World';
console.log(`Hello, ${name}!`);

// Run with:
// node hello.js
```

■ **Tip:** Save the file and run: node hello.js — you should see the output in your terminal.

### Understanding the REPL

Node.js includes an interactive **REPL** (Read-Eval-Print Loop) — a quick sandbox to test code. Just type **node** in your terminal (no filename).

```
$ node
> 2 + 2
4
> 'Hello'.toUpperCase()
'HELLO'
> .exit    // quit the REPL
```

# ■ Core Built-in Modules

Node.js ships with powerful built-in modules — no installation needed.

## The 'fs' Module — Working with Files

```
const fs = require('fs');

// Write a file
fs.writeFileSync('note.txt', 'Hello from Node.js!');

// Read a file
const content = fs.readFileSync('note.txt', 'utf8');
console.log(content);  // Hello from Node.js!

// Async version (non-blocking – preferred)
fs.readFile('note.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

## The 'path' Module — Cross-platform Paths

```
const path = require('path');

console.log(path.join('folder', 'sub', 'file.txt'));
// Windows: folder\sub\file.txt
// Mac/Linux: folder/sub/file.txt

console.log(path.extname('photo.jpg'));  // .jpg
console.log(path.basename('/home/user/app.js'));  // app.js
```

## The 'os' Module — System Information

```
const os = require('os');

console.log(os.platform());   // 'linux', 'darwin', 'win32'
console.log(os.hostname());   // your machine name
console.log(os.freemem());    // free memory in bytes
console.log(os.cpus().length); // number of CPU cores
```

■ **Tip:** Use require() to import any built-in module. No installation needed — they come with Node.js.

# ■ Building Your First Web Server

## Creating an HTTP Server

Node.js has a built-in **http** module that lets you create a web server in just a few lines. No frameworks needed for a basic server!

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  // Set response headers
  res.writeHead(200, { 'Content-Type': 'text/html' });

  // Send response based on URL
  if (req.url === '/') {
    res.end('Welcome to my Node.js server!');
  } else if (req.url === '/about') {
    res.end('About PageBuilt with Node.js');
  } else {
    res.writeHead(404);
    res.end('404 - Page Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

### How it works:

| Step | What happens |
|---|---|
| 1 | http.createServer() creates a server and gives you req (request) and res (response). |
| 2 | req.url contains the path the user visited (e.g., '/' or '/about'). |
| 3 | res.writeHead() sets the HTTP status code and headers. |
| 4 | res.end() sends the response body and closes the connection. |
| 5 | server.listen(3000) starts the server on port 3000. |

■ **Tip:** Run node server.js then open http://localhost:3000 in your browser to see it live!

# ■ npm — Node Package Manager

## What is npm?

**npm** is the world's largest software registry. It comes bundled with Node.js and lets you install, share, and manage packages (reusable code libraries) for your projects.

### Essential npm Commands

| Command | Description |
|---|---|
| `npm init -y` | Create package.json with defaults |
| `npm install express` | Install a package (saved to node_modules) |
| `npm install -D nodemon` | Install as dev dependency |
| `npm install` | Install all deps from package.json |
| `npm uninstall lodash` | Remove a package |
| `npm list` | Show installed packages |
| `npm outdated` | Check for outdated packages |
| `npm update` | Update packages to latest versions |
| `npm run start` | Run the 'start' script from package.json |

## Using Express — The Most Popular Node Framework

Install Express, then build a cleaner web server:

```
// Step 1: npm init -y && npm install express

// app.js
const express = require('express');
const app = express();

// Routes
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.get('/api/users', (req, res) => {
  res.json([{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }]);
});

app.listen(3000, () => console.log('Express server on port 3000'));
```

■ **Tip:** Express makes routing, middleware, and JSON APIs much simpler than the raw http module.

# ■ Async JavaScript & Next Steps

## Callbacks, Promises & Async/Await

Node.js is **asynchronous** by nature. Understanding async patterns is crucial.

### Async/Await (Modern — Recommended)

```
const fs = require('fs').promises;

async function readFile() {
  try {
    const data = await fs.readFile('note.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error('Error:', err.message);
  }
}

readFile();
```

## Quick Cheat Sheet

| Concept | Syntax / Example |
|---|---|
| **Import module** | `const fs = require('fs')` |
| **ES Module** | `import fs from 'fs'  (use .mjs or type:module)` |
| **Write file** | `fs.writeFileSync('f.txt', 'data')` |
| **Read file** | `const d = fs.readFileSync('f.txt','utf8')` |
| **HTTP server** | `http.createServer((req,res)=>{}).listen(3000)` |
| **Express route** | `app.get('/path', (req,res) => res.send('hi'))` |
| **JSON response** | `res.json({ key: 'value' })` |
| **URL params** | `app.get('/user/:id', (req,res) => req.params.id)` |
| **Middleware** | `app.use(express.json())` |
| **Environment** | `process.env.PORT \|\| 3000` |

## Your Learning Roadmap

| | |
|---|---|
| ■ **Done** | Node.js basics · npm · http module · Express intro |

| | |
|---|---|
| ■ **Next** | REST APIs · Middleware · MongoDB with Mongoose |
| ■ **Advanced** | Authentication (JWT) · WebSockets · Deployment (Heroku / Railway) |
| ■ **Resources** | nodejs.org/docs · expressjs.com · npmjs.com |